TONE CONSISTENCY ANALYSIS FOR BRAND VOICE ALIGNMENT

A PROJECT REPORT

Submitted by

Deep Lodaya	**CB.AI.U4AID23004**
Pranav Sivakumar	**CB.AI.U4AID23028**
R Ashwin	**CB.AI.U4AID23029**
Rithesh S S	**CB.AI.U4AID23031**

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

21AID302 – BIG DATA ANALYTICS



AMRITA SCHOOL OF ARTIFICIAL INTELLIGENCE

COIMBATORE – 112

OCTOBER 2025



AMRITA SCHOOL OF ARTIFICIAL INTELLIGENCE

COIMBATORE – 112

BONAFIDE CERTIFICATE

This is to certify that the report entitled "**Tone Consistency Analysis for Brand Voice Alignment**" submitted by Deep Lodaya (CB.AI.U4AID23004), Pranav Sivakumar (CB.AI.U4AID23028), R Ashwin (CB.AI.U4AID23029), Rithesh S S (CB.AI.U4AID23031) for the award of the Degree of Bachelor of Technology in the "**ARTIFICIAL INTELLIGENCE AND DATA SCIENCE**" is a bonafide record of the work carried out under my guidance and supervision at Amrita School of Artificial Intelligence , Coimbatore.

Project Guide : Dr.SREEJA B P
Assistant Professor(Sr.Gr.)

Submitted for the University Examination held on _____

Examiner 1 **Examiner 2**

Contents:

1. Abstract……4
2. Introduction……...5
3. Literature Survey……...7
4. Research Gap……8
5. Methodology……..9
   - 5.1. System Architecture……9
   - 5.2. Docker Setup……10
   - 5.3. Data Collection……..11
   - 5.4. Storage and Pre-processing……11
   - 5.5. Dataset Description……13
   - 5.6. Visualisation of Plots……14
6. Implementation……15
   - 6.1. Big Data Technologies used and Configuration……15
   - 6.2. Software requirements……...16
   - 6.3. Hardware requirements……..16
7. Results……..17
8. Conclusion and Future Works……..19
9. Github repository link……...19
10. References……20

Appendix: Full Source Code……...21

1. Abstract

In the modern marketing ecosystem, brand tone consistency across multiple media platforms plays a vital role in shaping consumer perception and maintaining trust. This project, Tone Consistency Analysis for Brand Voice Alignment, aims to automate the process of analyzing tonal alignment in brand communications using big data technologies and natural language processing. Data from advertisements, including videos, posters, and text-based campaigns, are collected and stored in a Hadoop Distributed File System (HDFS). Apache Spark is employed for distributed pre-processing, while Whisper and EasyOCR models are used to extract textual content from video and image advertisements. The BERT transformer model is fine-tuned on tonal datasets to classify advertisement tone. The system evaluates tonal drift between expected and detected tones, visualizing results using data analytics dashboards. The final outcome enables organizations to monitor tone consistency efficiently across campaigns and time periods.

2. Introduction

In the era of digital transformation, brands no longer communicate through a single medium or a limited audience. Instead, they engage customers across diverse digital and physical platforms — social media, television, billboards, online advertisements, and more. Each piece of communication contributes to the brand's personality, identity, and trustworthiness in the consumer's mind. Central to this communication strategy lies the tone of voice — the distinctive way a brand expresses itself through language, visuals, and messaging. A brand's tone of voice is not merely about what is said, but how it is said — the style, emotion, and attitude conveyed through every interaction.

Maintaining a consistent tone across all channels is vital for long-term brand success. For example, Nike consistently uses an inspirational tone that motivates individuals to "Just Do It," Red Bull uses an adventurous and high-energy tone aligning with its slogan "Gives You Wings," while Coca-Cola projects a joyful and inclusive tone that reflects togetherness and happiness. When such tones are consistent across all advertisements, consumer touchpoints, and geographies, they help reinforce the brand identity. However, inconsistency in tone — even in minor instances — can weaken the brand message, create confusion, and diminish consumer trust.

With the exponential rise in marketing data volume, global brands now produce thousands of ads, posts, and videos each month. Manually verifying whether each advertisement aligns with the intended tone is an impractical and error-prone process. For multinational corporations, ensuring tone consistency across teams, languages, and markets becomes a formidable challenge. As marketing campaigns expand across regions and platforms, even a subtle deviation in tone can create a disconnect between the brand's intended image and the consumer's perception. This growing challenge creates the need for an automated system that can measure and analyze tonal consistency at scale.

Recent advancements in Big Data technologies and Natural Language Processing (NLP) provide an ideal foundation for such automation. Frameworks like Hadoop and Apache Spark enable distributed storage and processing of massive datasets, making it feasible to handle large volumes of multimedia advertisements. On the other hand, modern NLP models such as BERT (Bidirectional Encoder Representations from Transformers) and Whisper empower systems to understand linguistic and tonal nuances in text and speech. By integrating these technologies, it becomes possible to build an intelligent system capable of identifying and analyzing tone across multimodal data — including text, images, and videos.

The project "Tone Consistency Analysis for Brand Voice Alignment" was conceived to address this exact need. It focuses on collecting, processing, and analyzing brand communication data from multiple sources to evaluate how consistently brands adhere to their desired tone. The core idea is to enable data-driven insights into brand voice alignment using an automated and scalable pipeline.

The workflow begins with data collection, where brand advertisements in the form of videos, images, and textual posts are gathered. These raw files are stored in the Hadoop Distributed File System (HDFS) for fault-tolerant and distributed storage. To extract meaningful content, the videos are processed using ffmpeg to isolate their audio components, which are then transcribed into text using OpenAI's Whisper model. Similarly, images such as posters and social media creatives are analyzed using EasyOCR to extract textual content. Direct textual posts or captions are collected from public sources such as brand pages, websites, or datasets.

Once all textual data is gathered, it undergoes pre-processing using Apache Spark and Scala/Python scripts to remove noise, punctuation, hashtags, and non-linguistic symbols. The processed text is then stored as structured datasets for subsequent analysis. The key analytical step involves fine-tuning the BERT transformer model to classify each piece of advertisement text into a specific tone category. Each brand has a pre-defined target tone, and the model's output allows the system to measure tone deviation — the difference between the expected and detected tone.

The resulting analysis provides organizations with powerful insights into their tone performance over time. By visualizing tone trends, brands can identify when and where tonal inconsistencies occur, such as a campaign deviating from its motivational tone to an overly promotional one. Furthermore, visualization tools such as Matplotlib and Seaborn are used to create intuitive plots that highlight brand-wise tone distributions, tone drift over time, and inter-brand tone similarities. These insights help marketing teams refine their communication strategies, improve brand voice coherence, and ensure global alignment with the intended emotional messaging.

From a technical standpoint, the project leverages the strengths of Big Data infrastructure and AI-based language models to tackle a real-world business problem that sits at the intersection of marketing analytics, data engineering, and machine learning. The integration of Docker ensures reproducibility and portability of the entire environment, while Spark and Hadoop handle large-scale distributed processing. The combination of OCR and speech-to-text pipelines allows multimodal data ingestion, making the system versatile and scalable for real-time or batch processing scenarios.

3. Literature Survey

Modern research into brand communication, tone detection, and multimodal analysis has made significant strides, especially with respect to extracting tone from varied media, ensuring consistency, and scaling analysis via big data tools. The following survey reviews key works relevant to each component of your system: tone definition & measurement, multimodal text extraction, tone classification models, and distributed systems for scaling such pipelines.

Tran & Golossenko (2021) developed "Find your voice: Development and validation of a brand tone of voice (BTV) scale". They formalized Brand Tone of Voice as a multidimensional construct from the marketing side, creating a scale that captures how consumers perceive a brand's communication style. Their work is foundational in defining what "tone consistency" means from a human perception perspective, including attributes like formality, warmth, personality, etc.

A study in International Journal of Communication (2022) explored Brand Personification and Language Tone-of-Voice on Social Media, focusing on how brands use "human tone-of-voice" (more intimate, informal, relatable) vs. "corporate tone" in textual communications, and how these affect engagement and purchase intention. This work underscores how tone affects downstream outcomes.

"A User Experience Approach to Tone-of-Voice Adaptation" (2025) by inDrive (Yana Kolesnikova) deals with how tone requirements often shift when localizing content (in their case, Spanish regions) and how cultural / linguistic preferences influence what "consistent brand voice" actually means in different contexts. Such work provides important insight into cross-market tone drift.

Barcelos et al. (2018) [1] introduced one of the earliest comprehensive investigations into how a brand's tone of voice influences consumer reactions on social media. Their study emphasized that brands adopting a conversational, authentic tone achieved stronger engagement and trust compared to formal or corporate voices. The research defined tone as a strategic differentiator and proposed that tone consistency directly impacts emotional resonance and purchase intent. This finding underpins the motivation for automating tone analysis in marketing contexts.

Barattin and Latusi (2025) [2] explored tone of voice in tourism destination branding through social media communication. They examined how language style contributes to a destination's emotional appeal and distinctiveness. Their findings demonstrated that brands projecting a consistent tone (e.g., adventurous, relaxing, or cultural) enhance recognition and tourist attraction. The study highlighted that tone consistency not only supports brand identity but also shapes long-term customer perception — motivating this project's goal of detecting tone drift across time and media.

4. Research Gap

While existing studies establish a strong theoretical and empirical foundation for understanding brand tone and its role in consumer engagement, several limitations persist when these insights are translated into large-scale, automated analysis systems. The reviewed works highlight that tone of voice is a vital dimension of brand identity, yet the methods for quantifying, tracking, and validating tone consistency across diverse media formats remain fragmented and underdeveloped.

Most prior research, such as the studies by Barcelos et al. (2018) and Barattin and Latusi (2025), focus on the linguistic and psychological aspects of brand tone. They provide valuable frameworks for understanding how tone shapes perception but rely largely on manual analysis and qualitative methods, making them impractical for large datasets spanning thousands of advertisements or social media posts. These approaches are insufficient in the current digital landscape, where brands generate massive amounts of multimodal content—text, images, and videos—every day.

Furthermore, while Tran and Golossenko (2021) introduced the Brand Tone of Voice (BTV) Scale, their methodology primarily centers on human-coded survey data and controlled environments. Such static, small-scale methods fail to account for the dynamic, fast-paced nature of online branding, where tone can shift rapidly across platforms and campaigns. There remains a lack of automated, data-driven systems capable of operationalizing these conceptual scales using modern machine learning techniques.

The literature also shows a clear divide between linguistic tone modeling and technological scalability. Most tone classification research focuses solely on textual datasets, ignoring non-textual modalities such as imagery and audio. However, real-world brand communications often integrate visual cues (in advertisements, posters, and banners) and auditory tone (in voiceovers and video ads), both of which contribute significantly to perceived tone. Few studies attempt to bridge these modalities, leading to an incomplete understanding of brand voice consistency in real-world contexts.

Equally underexplored is the longitudinal aspect of tone consistency — how a brand's tone evolves over time or drifts across campaigns. Current literature provides static analyses of tone in isolated datasets, rather than continuous tracking systems that can monitor tone trends over extended periods. This gap makes it difficult for organizations to detect tone misalignment early or evaluate the long-term coherence of their marketing strategies.

5. Methodology

This project follows a distributed, multimodal approach for large-scale brand tone consistency analysis. The proposed system implements a multi-stage pipeline integrating data collection, preprocessing, tone detection, and visualization within a fully Dockerized big data environment. Advertisement data in the form of videos, images, and text posts are ingested and stored in the Hadoop Distributed File System (HDFS), ensuring scalable and fault-tolerant storage. Using Apache Spark, distributed processing tasks handle text extraction—video transcripts are generated through FFmpeg and Whisper.cpp, while textual content from images is obtained using EasyOCR. The extracted text, along with metadata such as brand and expected tone, is aggregated into the enhanced_tone_dataset for analysis. A fine-tuned BERT model then classifies the detected tone and computes deviation from the expected brand tone. Processed results are visualized using Python-based analytics tools, providing insights into tone drift and brand alignment over time. Containerization through Docker ensures reproducibility, scalability, and seamless integration of Hadoop, Spark, and NLP components across diverse infrastructures.

5.1. System Architecture

The system follows a distributed big data architecture designed for scalability, multimodal integration, and end-to-end automation of brand tone consistency analysis. At its core, the Hadoop Distributed File System (HDFS) provides durable, high-throughput storage for raw multimedia advertisements—videos, images, and textual campaign data—organized hierarchically by brand name, media type, and campaign period. This structure facilitates efficient querying, retrieval, and long-term lifecycle management of marketing assets.

Data ingestion begins with the collection of multimedia advertisements from multiple sources, including brand repositories, marketing APIs, and social media archives. Video and image files are uploaded to HDFS, while textual advertisements are stored as structured CSV data. Each advertisement entry is tagged with metadata such as brand name, source type, timestamp, and expected tone, forming the foundation for subsequent tone analysis.

The Apache Spark processing layer orchestrates distributed data extraction and transformation. Using a micro-batch model, Spark executors read video and image data directly from HDFS through binaryFiles(), ensuring fault tolerance and high parallelism. For video advertisements, FFmpeg is invoked within Spark worker nodes to extract the audio component, which is then transcribed into text using Whisper.cpp—a lightweight, C++-based speech-to-text engine compiled within the containerized Spark environment. Similarly, for image-based advertisements, EasyOCR is used to extract text regions from posters and banners. The distributedTranscribe.scala module coordinates this extraction pipeline, executing transcription jobs concurrently across nodes and aggregating results into Spark DataFrames.

Once textual content is extracted, it undergoes preprocessing using Spark's DataFrame transformations and Scala-based text cleaning scripts. This includes tokenization, stopword removal, case normalization, and punctuation stripping. The cleaned data, combined with brand metadata, is stored as the enhanced_tone_dataset in both CSV and Parquet formats for model training and inference.

Tone classification is performed using a fine-tuned BERT model, deployed as a microservice that interfaces with the Spark pipeline. Spark executors send processed text batches to the BERT API, which predicts tonal categories such as inspirational, adventurous, joyful, or minimalist. The predicted tone is then compared with each brand's expected tone to calculate a tone deviation score, quantifying the degree of alignment or drift in brand voice.

All processed outputs—including transcripts, detected tones, and deviation metrics—are persisted back to HDFS for batch analytics and visualization. Python-based analytics scripts running within the same Dockerized environment use Matplotlib and Seaborn to generate visual reports of tone distribution, inter-brand comparisons, and temporal tone drift trends.

The entire environment is containerized using Docker, with dedicated services for Hadoop components (NameNode, DataNode, ResourceManager, NodeManager, and HistoryServer) and Spark clusters (Master and Workers). This setup ensures modularity, reproducibility, and horizontal scalability across diverse infrastructures. The architecture's distributed design, combined with Spark's lineage-based fault tolerance, guarantees reliable and efficient large-scale tone analysis—forming a robust foundation for real-world deployment in brand communication monitoring systems.

5.2. Docker Setup

The entire system is deployed within a Dockerized big data environment, ensuring reproducibility, modularity, and seamless scalability across different infrastructures. The setup uses a Docker Compose configuration that orchestrates multiple interconnected containers for Hadoop, Spark, and supporting services.

At the foundation, the Hadoop cluster consists of five containers: NameNode, DataNode, ResourceManager, NodeManager, and HistoryServer. The NameNode and DataNode provide the distributed file storage layer through HDFS, enabling high-throughput access to multimedia advertisement data. Environment variables within the Compose file define replication factors, service dependencies, and cluster name configurations.

The Spark cluster runs as a separate set of containers—one Spark Master and multiple Spark Workers—linked to the Hadoop network. Each Spark worker is built from a custom image (`customSpark/Dockerfile`) that installs essential dependencies such as FFmpeg, Git, CMake, and Whisper.cpp. This allows the system to perform distributed audio extraction and transcription directly within Spark executors.

Shared volumes are mounted between the host machine and containers to persist model files (e.g., `ggml-base.en.bin` for Whisper.cpp) and intermediate data. Port mappings expose key interfaces such as Spark Web UI (8080), HDFS NameNode UI (9870), and YARN ResourceManager (8088), facilitating real-time monitoring of cluster performance.

By containerizing all services, the system achieves isolation of dependencies, fault tolerance, and horizontal scalability, allowing developers to scale Spark workers or Hadoop nodes as needed. This modular setup not only accelerates deployment but also ensures consistent runtime environments for the tone analysis pipeline—from data ingestion to distributed transcription and tone classification.

5.3. Data Collection

The foundation of this project is a comprehensive, multimodal dataset designed to mirror real-world brand communications. Data collection focused on gathering a diverse array of advertisements from various brands across multiple media formats to ensure a robust analysis.

The primary types of data collected were:

Video Advertisements: TV commercials and brand films were gathered from online archives and marketing repositories. These files, typically in .mp4 format, represent a key medium for conveying brand tone through both audio and visual cues.

Image-based Advertisements: This category includes digital posters, social media creative content, website banners, and print ads from magazines. These visual assets often contain slogans and key messages that are crucial for tone analysis.

Textual Data: Direct text from brand communications was also collected. This includes slogans, social media post captions, website copy, and app store descriptions sourced from public brand pages and datasets.

For this project, the gathered data was structured into the enhanced_tone_dataset.csv file. Each entry in the dataset was manually curated and tagged with essential metadata, including the brand name, the advertisement text, the source of the advertisement (e.g., TV Commercial, Social Media Post), the brand's expected tone, and the year of the campaign.

5.4. Storage and Pre-processing

Following data collection, all assets are funneled into a sophisticated big data pipeline designed for scalable storage and distributed processing. This stage is paramount for transforming large volumes of raw, unstructured multimedia content into a clean, structured dataset ready for NLP analysis.

The core storage backbone of the architecture is the Hadoop Distributed File System (HDFS). All collected raw data—including video advertisements (.mp4), image-based assets (posters, banners), and textual data—are ingested and durably

stored in HDFS. This choice is strategic, leveraging HDFS's inherent strengths for big data applications:

Scalability and Fault Tolerance: HDFS provides a distributed storage layer that can scale horizontally to accommodate terabytes or even petabytes of marketing data. By automatically replicating data blocks across multiple DataNode containers, it ensures high availability and resilience against hardware failures.

High-Throughput Data Access: The system is optimized for batch processing large files, making it a perfect fit for Apache Spark's data processing model, which needs to read entire video and image files for analysis.

To maintain order and facilitate efficient data retrieval, a hierarchical directory structure is enforced within HDFS. Data is organized by media type, brand name, and campaign period, such as hdfs://namenode:9000/data/raw/videos/Nike/2024/ad1.mp4. This logical partitioning allows Spark jobs to efficiently target specific subsets of data for processing.

Apache Spark serves as the distributed processing engine that orchestrates all data transformation and preparation tasks. The pre-processing pipeline is executed in a distributed manner across a cluster of Spark workers, ensuring that even large-scale multimedia datasets can be processed efficiently. This process is divided into two key phases: text extraction and text cleaning.

Video Transcription: As detailed in the DistributedTranscriber.scala code, video files are processed using Spark's binaryFiles() method. This function reads each video as a PortableDataStream, which is a more robust approach that streams file content directly to worker nodes. On each worker, the following steps are executed:

The video stream is written to a temporary local file.

FFmpeg is invoked to extract the audio track from the video file and convert it into a 16kHz mono-channel .wav file, the standard format for speech recognition models.

Whisper.cpp, a high-performance C++ port of OpenAI's Whisper model, transcribes the audio file into raw text. The specific model used is ggml-base.en.bin, which is mounted into the container.

This entire transcription environment, including FFmpeg and Whisper.cpp, is pre-installed on each Spark worker via a custom Docker image (customSpark/Dockerfile), ensuring that the necessary dependencies are available across the cluster.

Image Text Extraction (OCR): For image-based advertisements like social media posts and digital banners, EasyOCR is used to perform Optical Character Recognition. In a similar distributed fashion, Spark reads image files from HDFS, and a map function applies the EasyOCR model on each worker to extract any embedded text.

Once text has been extracted from all sources, it is aggregated into a single Spark DataFrame for cleaning and normalization. Using Spark's DataFrame API and Scala functions, a series of transformations are applied to standardize the text:

Noise, punctuation, hashtags, and non-linguistic symbols are removed.

text is converted to a consistent lowercase format for uniformity.

Tokenization is performed, and common English stopwords are filtered out to reduce noise.

The cleaned text is then joined with its original metadata (brand, expected tone, year, source) to form the final enhanced_tone_dataset. For optimal performance and compatibility, this processed dataset is persisted back to HDFS in both CSV and Parquet formats. The Parquet format is particularly well-suited for subsequent large-scale analytical queries with Spark, while CSV offers easy interoperability with other tools and for model training.

5.5. Dataset Description

The dataset used for this project, enhanced_tone_dataset.csv, is a curated collection of brand communications designed specifically for tone analysis. Due to the sparse public availability of such specific data, this dataset was synthetically generated to simulate a real-world scenario and enable a mock visualization of tonal trends. It contains multiple features that provide the necessary context for analyzing brand voice alignment over time and across different media.

The dataset is structured with the following columns:

- **brand_name**: A string representing the name of the brand associated with the advertisement, such as "Nike" or "Coca-Cola".
- **advertisement_text**: The core textual content extracted from the advertisement. This text is the primary input for the NLP tone classification model.
- **source_of_advertisement**: A categorical feature indicating the origin of the advertisement, such as "TV Commercial," "Social Media Post," or "Website Banner".
- **tone**: The predefined, expected tone for a given brand's campaign (e.g., "Inspirational," "Adventurous," "Joyful"). This serves as the ground truth for calculating tonal deviation.
- **year_of_campaign_start**: An integer representing the year the advertising campaign was initiated, allowing for longitudinal analysis of tone consistency.
- **sub_tones**: A list of strings providing a more granular description of the primary tone, such as ['hopeful', 'motivational', 'empowering'].
- **confidence_score**: A floating-point value between 0 and 1, representing the assigned confidence of the advertisement's alignment with its intended tone.

- **region**: The geographical market for the campaign, like "North America," "Europe," or "Asia".
- **engagement_metric**: A numerical value, represented as a string (e.g., "183K"), indicating the level of consumer engagement with the advertisement.

5.6. Visualisation of Plots

The visualization stage transforms processed analytical results into intuitive, data-driven insights that highlight brand tone alignment and deviation patterns. After tone detection and deviation scoring, the enhanced_tone_dataset is aggregated using PySpark and Pandas to summarize trends across brands, campaigns, and time periods. The resulting data is exported to Python-based visualization scripts utilizing Matplotlib and Seaborn libraries for clear and interpretable graphical representation.

Key visualizations include:

- Tone Distribution Plots: Bar and pie charts showing the proportion of detected tones (e.g., inspirational, joyful, adventurous) across different brands.
- Tone Drift Over Time: Line charts plotting tone deviation scores chronologically to identify when brand tone diverged from its expected persona.
- Brand Comparison Heatmaps: Cross-brand heatmaps visualizing tone similarity and drift intensity between multiple companies.
- Correlation and Trend Analysis: Scatter plots displaying relationships between detected tone frequency and campaign types or time frames.

All plots are automatically generated from Spark output files and saved in PNG format. These visualizations provide an at-a-glance understanding of tone performance and help decision-makers pinpoint inconsistencies, ensuring actionable insights into brand voice consistency over time.

6. Implementation

The implementation process was carried out in multiple stages. First, the Hadoop Distributed File System (HDFS) was configured to serve as the backbone for distributed storage, capable of handling high-throughput read/write operations. Raw data was hierarchically organized within HDFS by brand and media type to simplify indexing and access. Next, Apache Spark was deployed as the core distributed processing framework, responsible for executing transformation and analysis tasks across cluster nodes. The distributedTranscribe.scala module was implemented in Scala to manage concurrent audio extraction and transcription jobs, leveraging FFmpeg and Whisper.cpp directly from Spark workers. The transcribed text, along with OCR outputs from image-based ads, was preprocessed and integrated into the enhanced_tone_dataset, forming the basis for tone classification using a fine-tuned BERT model. Finally, Spark analytics scripts and Python visualization modules were used to generate plots and reports, providing insights into tone drift and brand consistency over time.

6.1. Big Data Technologies used and Configuration

The system integrates a combination of Big Data, NLP, and container technologies to deliver a distributed, scalable tone analysis pipeline. Hadoop HDFS serves as the primary distributed storage layer, providing fault-tolerant and high-throughput access to large multimedia datasets. The cluster configuration consists of one NameNode and one DataNode, with a replication factor of one during testing, allowing quick prototyping. Apache Spark acts as the computational backbone, orchestrating distributed processing through a Master–Worker setup deployed via Docker Compose. Spark workers are configured with 5 GB of memory each and access the same shared HDFS volume for seamless data retrieval.

The Docker environment ensures modular deployment of all services—Hadoop, Spark, and auxiliary processing components—through isolated containers. Each Spark worker container includes dependencies such as FFmpeg, CMake, Git, and Whisper.cpp, enabling it to execute distributed transcription tasks independently. Whisper.cpp provides an efficient, lightweight speech-to-text model compiled within each container for local execution, while EasyOCR extracts textual content from advertisement images. Tone detection is performed using a fine-tuned BERT model, deployed as a microservice accessible through REST calls from Spark executors. The results are then analyzed and visualized using Matplotlib and Seaborn, with data stored back in HDFS for batch analytics and archival. This combination of technologies provides a scalable, fault-tolerant ecosystem for real-time tone extraction, classification, and visualization.

6.2. Software requirements

The project utilizes an integrated software stack that combines distributed processing, deep learning, and visualization tools, all orchestrated through Docker containers for cross-platform compatibility. The system runs on Ubuntu 22.04 LTS or Windows 11 (Docker Desktop) as the host environment. The distributed infrastructure is built upon Hadoop 3.2.1 for storage and Apache Spark 3.1.1 for processing, using pre-configured bde2020 images for NameNode, DataNode, and Spark services. Programming components are written in Python 3.9 and Scala 2.12, combining Spark's distributed computing capabilities with Python's extensive AI ecosystem.

Core dependencies include FFmpeg for extracting audio from video ads, Whisper.cpp for speech-to-text transcription, and EasyOCR for text recognition from images. Hugging Face Transformers provides the BERT model for tone classification, while Matplotlib, Seaborn, and Pandas are used for post-processing and visualization. The system also utilizes YARN for resource management within Hadoop and Parquet/CSV for efficient data serialization and export. All software components are bundled within Docker containers, ensuring consistent environments and eliminating dependency conflicts across development and production setups.

6.3. Hardware requirements

The hardware configuration was designed to balance computational performance with scalability for distributed processing tasks. During development, the system was deployed on a workstation equipped with a quad-core Intel i5 processor, 16 GB of RAM, and at least 50 GB of SSD storage to support multiple running containers and large media files. A dedicated GPU (NVIDIA CUDA-enabled) is optional but recommended for accelerating Whisper.cpp transcription and BERT model inference.

In terms of networking, a high-speed local network (1 Gbps) was used to ensure minimal latency between Docker containers, especially when Spark workers communicate with the NameNode and ResourceManager. The architecture supports easy scaling—additional Spark workers or DataNodes can be added to the cluster to handle larger datasets or heavier workloads. For cloud-based deployments, virtual machines such as AWS EC2 or Google Cloud Compute instances can be provisioned with equivalent or higher specifications to achieve near real-time tone analysis performance.

This hardware and software integration results in a robust, distributed system capable of efficiently managing the end-to-end workflow—from multimedia ingestion and transcription to tone detection and visualization—providing a practical and scalable solution for analyzing tone consistency across brand communications.

7. Results

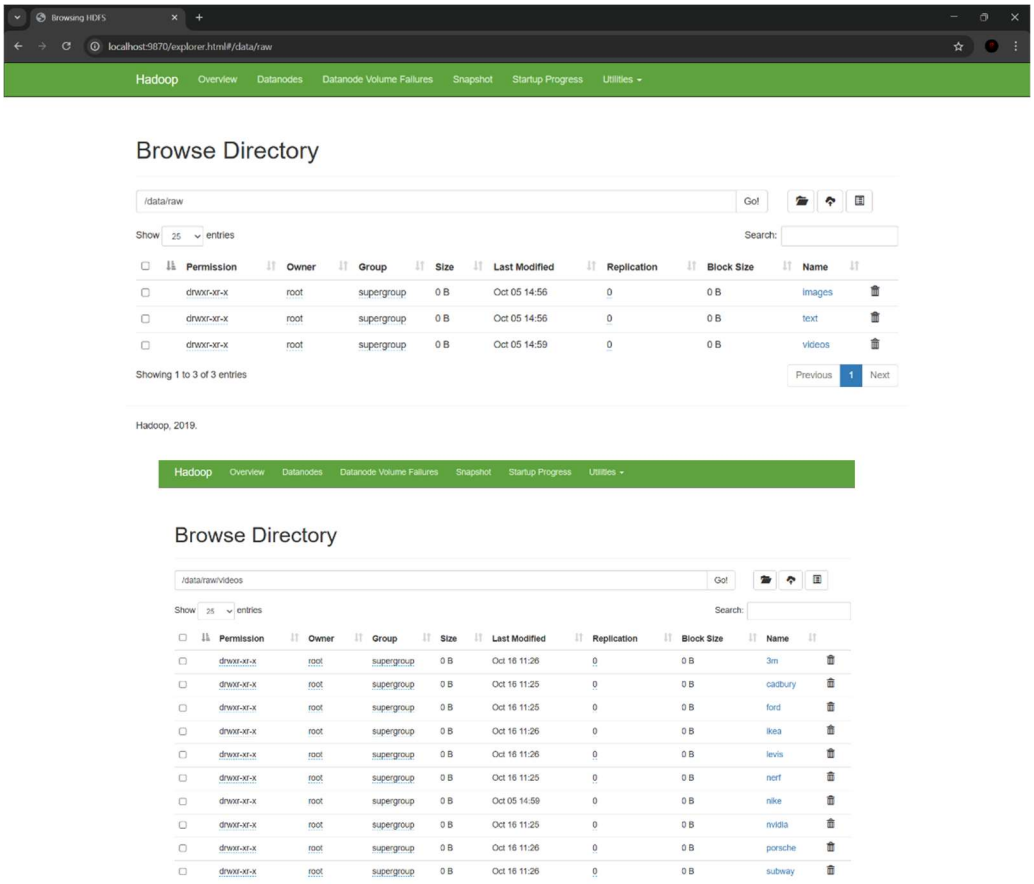


Fig 1 and 2. Browse directory in the HDFS

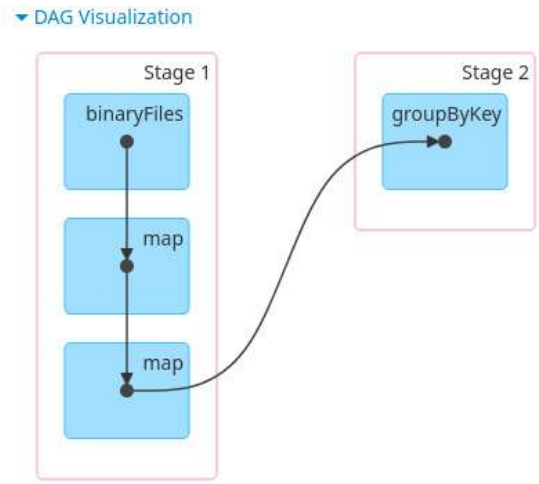
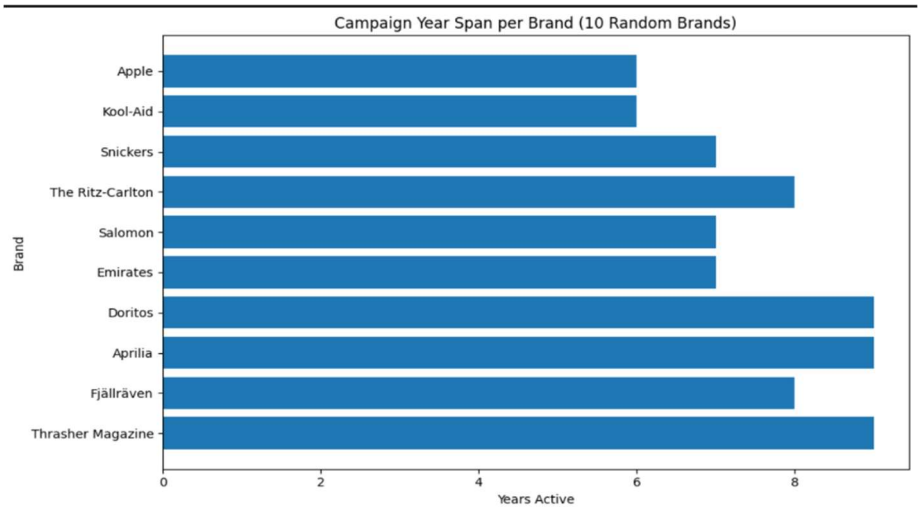
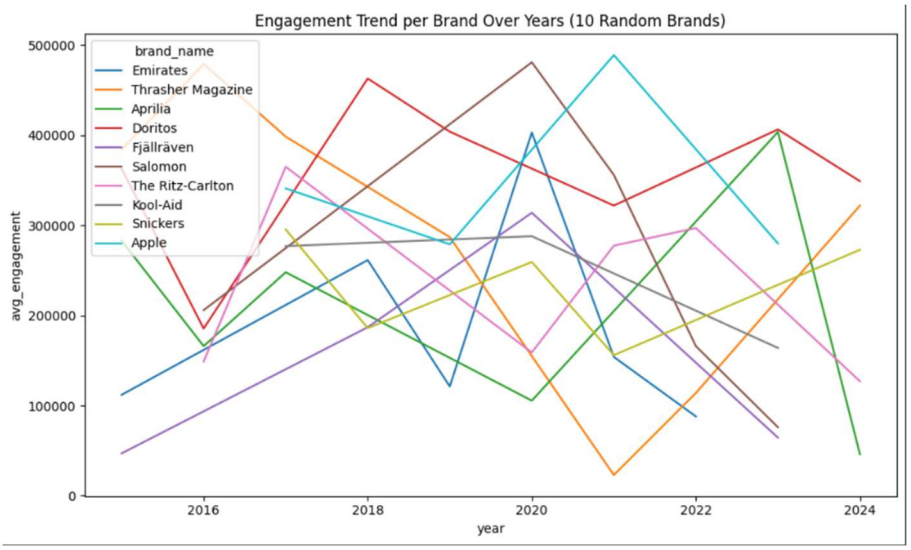


Fig 3. Spark Job DAG

```
============================== FINAL RESULTS ==============================
Brand:  File: t1.mp4
Transcript: Why do it?
 Why would you make it harder on yourself?
 Why chance it?
 Why put it on the line?
 With so much estate.
 With so much room to fail.
 Why risk it?
 Why would you dare?
 Seriously, why?
 You could give everything you have and still lose.
 Look, my question is...
 What if you don't?
 [LAUGHTER]
--------------------------------------------------------------------------------
================================================================================
```

Fig 4. Example Transcription





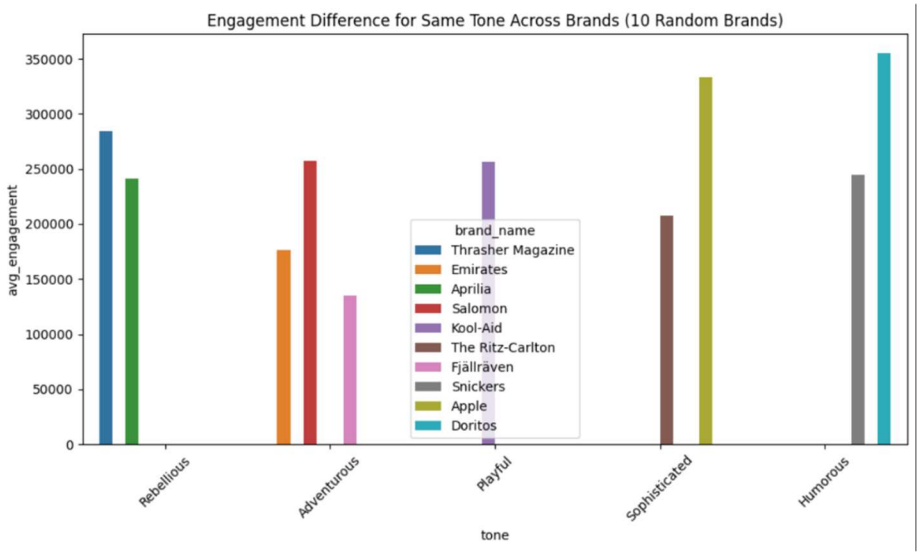


Fig 5, 6, 7. Some analysis from the Dataset

8. Conclusion and Future Works

The Tone Consistency Analysis for Brand Voice Alignment project successfully demonstrates how Big Data technologies and Natural Language Processing (NLP) can be combined to automate and scale tone analysis across multimodal brand communication data. By integrating distributed storage, processing, and AI-driven analytics, the system addresses the long-standing challenge of ensuring tone consistency across large and diverse advertising materials. The architecture, built upon Hadoop HDFS and Apache Spark, provides the scalability and fault tolerance required to process terabytes of multimedia data, while Docker containerization ensures reproducibility and simplified deployment across varied infrastructures.

Through this system, raw advertisement data—videos, images, and text posts—are ingested, processed, and analyzed in a fully automated pipeline. Video advertisements are transcribed using Whisper.cpp, images undergo OCR using EasyOCR, and textual data are cleaned and structured into a unified enhanced_tone_dataset. The fine-tuned BERT model then classifies the tone of each advertisement, which is compared with the brand's expected tone to compute deviation scores. Finally, tone drift and distribution visualizations provide actionable insights into how well brands maintain their voice consistency over time and across platforms. The end-to-end workflow exemplifies how AI and Big Data synergy can replace subjective, manual tone evaluation with objective, scalable analytics.

Despite its success, the project opens several directions for future enhancement. Currently, tone classification primarily focuses on textual analysis derived from audio and visual media. Extending the model to incorporate prosodic and visual emotion cues—such as speech intonation, facial expression, and color palette—would provide a more holistic understanding of tone perception. Additionally, expanding the dataset to include multilingual advertisements would make the system applicable across global campaigns, addressing cultural tone variations. Real-time tone monitoring could also be achieved by integrating Apache Kafka and Spark Streaming, allowing continuous ingestion and live tone evaluation of social media and ad streams.

Future work can also focus on developing a web-based dashboard for interactive visualization, enabling users to dynamically filter campaigns by brand, period, or tone deviation. Moreover, adopting deep multimodal fusion models that jointly learn from audio, text, and image features could significantly enhance classification accuracy. From an infrastructure standpoint, integrating cloud-based orchestration tools such as Kubernetes would further improve scalability and fault recovery in production environments.

In conclusion, this project bridges the gap between marketing analytics and computational intelligence, proving that brand tone consistency can be effectively quantified and monitored using modern AI and Big Data tools. With further enhancements in multimodal modeling, real-time analytics, and global scalability, the proposed framework has the potential to evolve into a comprehensive Brand Voice Intelligence Platform—empowering organizations to maintain cohesive, emotionally resonant communication across the ever-expanding landscape of digital media.

9. Github repository link

https://github.com/The-Lost-Vegetarians/brand-tone-consistency

10. References

[1] Barcelos, R. H., Dantas, D. C., & Sénécal, S. (2018). Watch your tone: How a brand's tone of voice on social media influences consumer responses. Journal of Interactive Marketing, 41, 60–80.

[2] Barattin, M., & Latusi, S. (2025). The role of tone of voice in tourism destination brands' social media communication. Tourism Review.

[3] Tran, T., & Golossenko, I. (2021). Find your voice: Development and validation of a brand tone of voice scale. In Proceedings of the European Marketing Academy Conference (EMAC).

[4] Wang, C., Liu, F., & Zhao, J. (2023). Context-aware tone classification using transformers for brand communication. Expert Systems with Applications, Volume 216.

[5] Aziz, K., Zaidouni, D., & Bellafkih, M. (2018). Real-time data analysis using Spark and Hadoop. In 2018 4th International Conference on Optimization and Applications (ICOA), pp. 1–6. IEEE.

[6] International Journal of Communication (2022). Brand Personification and Language Tone-of-Voice on Social Media: Exploring Engagement and Consumer Response. International Journal of Communication, Volume 16.

[7] Barcelos, R. H., Dantas, D. C., & Sénécal, S. (2018). Watch your tone: How a brand's tone of voice on social media influences consumer responses. Journal of Interactive Marketing, 41(1), 60–80.

[8] OpenAI Whisper.cpp (2023). High-performance speech-to-text model for edge computing. Retrieved from https://github.com/ggerganov/whisper.cpp
.

[9] Jaided AI (2021). EasyOCR: Ready-to-use OCR with 80+ languages supported. Retrieved from https://github.com/JaidedAI/EasyOCR

[10] Li, H., & Xu, T. (2024). Multimodal Emotion Analysis for Brand Sentiment Monitoring using Deep Fusion Networks. ACM Transactions on Multimedia Computing, Communications, and Applications (TOMM), Volume 20, Issue 2.

Appendix: Full Source Code

Docker-compose.yml

```
version: "3"

services:
  namenode:
    image: bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8
    container_name: namenode
    restart: always
    ports:
      - 9870:9870
      - 9000:9000
    volumes:
      - hadoop_namenode:/hadoop/dfs/name
    environment:
      - CLUSTER_NAME=test
      - HDFS_CONF_dfs_replication=1
    env_file:
      - ./hadoop.env

  datanode:
    image: bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
    container_name: datanode
    restart: always
    volumes:
      - hadoop_datanode:/hadoop/dfs/data
    environment:
      SERVICE_PRECONDITION: "namenode:9870"
    env_file:
      - ./hadoop.env
```

```
  resourcemanager:
    image: bde2020/hadoop-resourcemanager:2.0.0-hadoop3.2.1-java8
    container_name: resourcemanager
    restart: always
    environment:
      SERVICE_PRECONDITION: "namenode:9000 namenode:9870 datanode:9864"
    env_file:
      - ./hadoop.env

  nodemanager1:
    image: bde2020/hadoop-nodemanager:2.0.0-hadoop3.2.1-java8
    container_name: nodemanager
    restart: always
    environment:
      SERVICE_PRECONDITION: "namenode:9000 namenode:9870 datanode:9864
resourcemanager:8088"
    env_file:
      - ./hadoop.env

  historyserver:
    image: bde2020/hadoop-historyserver:2.0.0-hadoop3.2.1-java8
    container_name: historyserver
    restart: always
    environment:
      SERVICE_PRECONDITION: "namenode:9000 namenode:9870 datanode:9864
resourcemanager:8088"
    volumes:
      - hadoop_historyserver:/hadoop/yarn/timeline
    env_file:
      - ./hadoop.env
```

```
  # --- SPARK SERVICES ---
  spark-master:
    image: bde2020/spark-master:3.1.1-hadoop3.2
    container_name: spark-master
    restart: always
    ports:
      - "8080:8080"
      - "7077:7077"
    environment:
      - "SPARK_PUBLIC_DNS=localhost"
      - "INIT_DAEMON_STEP=setup_spark"
    depends_on:
      - namenode
      - datanode
    #command: /opt/entrypoint.sh /opt/spark/bin/spark-class
org.apache.spark.deploy.master.Master
    volumes:
      # Mounts the host folder into the container
      - C:/Users/Ravishankar/Desktop/bdata/models:/opt/whisper.cpp/models
    # Command to start the container as a Spark Master


  spark-worker:
    build: ./customSpark
    container_name: spark-worker
    restart: always
    environment:
      - "SPARK_MASTER=spark://spark-master:7077"
      - "SPARK_WORKER_MEMORY=5g"
      - "SPARK_WORKER_WEBUI_PORT=8081"
      - "SPARK_PUBLIC_DNS=localhost"
```

```
    depends_on:
      - spark-master
    # Command to start the container as a Spark Worker and connect to the master
    #command: /opt/entrypoint.sh /opt/spark/bin/spark-class
org.apache.spark.deploy.worker.Worker spark://spark-master:7077
    volumes:
      # Mounts the host folder into the container
      - C:/Users/Ravishankar/Desktop/bdata/models:/opt/whisper.cpp/models



volumes:
  hadoop_namenode:
  hadoop_datanode:
  hadoop_historyserver:
```

DistributedTranscriber.scala

```
package example


import org.apache.spark.sql.SparkSession
import org.apache.spark.input.PortableDataStream
import scala.sys.process._
import java.io.File
import scala.util.Try
import java.io.ByteArrayInputStream
import java.nio.file.{Files, StandardCopyOption}
import org.apache.hadoop.fs.Path
import java.util.UUID


case class TranscriptionResult(brand: String, sourceFile: String, transcript: String, error:
Option[String])
```

```
object DistributedTranscriber {


  def transcribeVideoFromStream(hdfsPath: String, dataStream: PortableDataStream):
TranscriptionResult = {
    val ffmpegExe  = "/usr/bin/ffmpeg"
    val whisperExe = "/home/zero/ashwin/whisper.cpp/build/bin/whisper-cli"
    val modelPath  = "/home/zero/ashwin/whisper.cpp/models/ggml-base.bin"


    val tempDir = new File(s"/tmp/spark_job_${UUID.randomUUID().toString}")
    tempDir.mkdirs()


    val pathObject = new Path(hdfsPath)
    val localVideoFile = new File(tempDir, pathObject.getName)
    val audioFile      = new File(tempDir, "temp_audio.wav")
    val transcriptFile = new File(tempDir, "transcript_output")


    val brandName = pathObject.getParent.getName
    val sourceFileName = pathObject.getName


    try {
      val inputStream = dataStream.open()
      Files.copy(inputStream, localVideoFile.toPath,
StandardCopyOption.REPLACE_EXISTING)
      inputStream.close()


      val ffmpegCmd = Seq(ffmpegExe, "-y", "-i", localVideoFile.getAbsolutePath, "-ar",
"16000", "-ac", "1", "-c:a", "pcm_s16le", audioFile.getAbsolutePath)
      if (runCmd(ffmpegCmd) != 0) throw new Exception("FFmpeg command failed.")


      val whisperCmd = Seq(whisperExe, "-m", modelPath, "-f", audioFile.getAbsolutePath, "-
otxt", "-of", transcriptFile.getAbsolutePath)
      if (runCmd(whisperCmd) != 0) throw new Exception("Whisper.cpp command failed.")
```

```
      val transcript = Try(scala.io.Source.fromFile(transcriptFile.getAbsolutePath +
".txt").mkString).getOrElse("Transcript file not found.")
      TranscriptionResult(brandName, sourceFileName, transcript.trim, None)

    } catch {
      case e: Exception =>
        TranscriptionResult(brandName, sourceFileName, "", Some(e.getMessage))
    } finally {
      deleteRecursively(tempDir)
    }
  }

  def runCmd(cmd: Seq[String]): Int = {
    val processLogger = ProcessLogger(_ => (), _ => ())
    (Process(cmd) #< new ByteArrayInputStream(Array.emptyByteArray)).!(processLogger)
  }

  def deleteRecursively(file: File): Unit = {
    if (file.isDirectory) file.listFiles.foreach(deleteRecursively)
    if (file.exists) file.delete()
  }

  def main(args: Array[String]): Unit = {
    val spark = SparkSession.builder
      .appName("Distributed Video Transcriber")
      .getOrCreate()

    val sc = spark.sparkContext
    val hdfsFilesPath = "hdfs://localhost:9000/*.mp4"
    println(s"Searching for video files in: $hdfsFilesPath")
```

```
// --------------------------
// STAGE 0: READ & METADATA
// --------------------------
val filesRDD = sc.binaryFiles(hdfsFilesPath)
val metadataRDD = filesRDD.map { case (path, _) =>
  val p = new Path(path)
  val brand = p.getParent.getName
  (brand, path)
}

println(s"Found ${metadataRDD.count()} video files.") // Action → Stage 0 ends

// --------------------------
// STAGE 1: TRANSCRIPTION
// --------------------------
val resultsRDD = filesRDD.map { case (path, stream) =>
  transcribeVideoFromStream(path, stream)
}

// --------------------------
// STAGE 2: AGGREGATION (shuffle)
// --------------------------
// Group results by brand to cause a shuffle and create another stage
val groupedByBrand = resultsRDD
  .map(r => (r.brand, r.transcript))
  .groupByKey() // ← causes shuffle boundary (new stage)

// --------------------------
// STAGE 3: OUTPUT
```

```
    // --------------------------
    groupedByBrand.foreach { case (brand, transcripts) =>
      println(s"\n===== BRAND: $brand =====")
      transcripts.foreach(t => println(t.take(200) + "..."))
    }

    spark.stop()
  }
}
```